Property based testing with Hypothesis

The-Compiler/hypothesis-talk

Freya Bruhin

July 15th, 2026







About me

The-Compiler/hypothesis-talk



Freya Bruhin / “The Compiler”

she/her or they/them

https://bruhin.software

freya@bruhin.software

About me

The-Compiler/hypothesis-talk

Freya Bruhin / "The Compiler"

she/her or they/them

https://bruhin.software

freya@bruhin.software

2011:



About me

The-Compiler/hypothesis-talk

Freya Bruhin / "The Compiler"

she/her or they/them

https://bruhin.software

freya@bruhin.software

2011:



2013, 2015:

pytest

About me

The-Compiler/hypothesis-talk

Freya Bruhin / "The Compiler"

she/her or they/them

https://bruhin.software

freya@bruhin.software

2011:



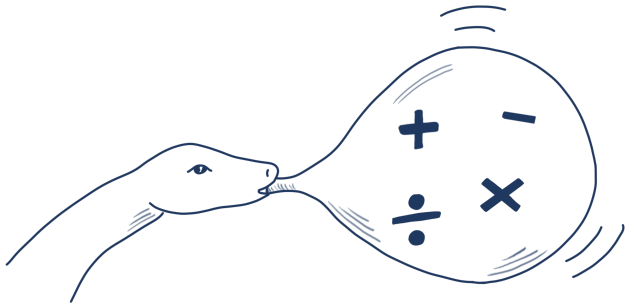
2013, 2015:

pytest

2019, 2020:

BRUHIN
SOFTWARE

The calculator example



Calculator project

rpncalc/utils.py

```
def calc(a, b, op):
    if op == "+":
        return a + b
    elif op == "-":
        return a - b
    elif op == "*":
        return a * b
    elif op == "/":
        return a / b
    raise ValueError("Invalid operator")
```

Reverse Polish Notation (RPN)

History

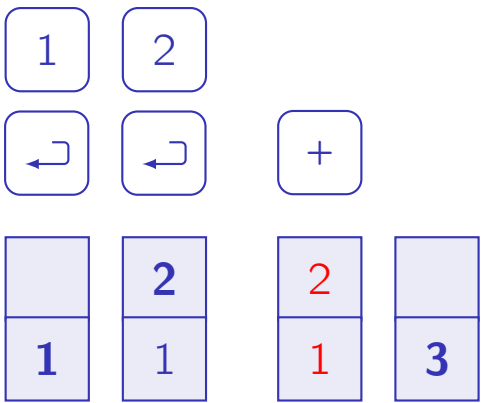


- Using a calculator without needing a = key, and without parentheses
- Makes it much easier to implement, using a stack data structure
- Used by all HP calculators in the 1970s–80s, still used by some today
- Displayed here: HP 12C financial calculator, introduced in 1981, still in production today (HPs longest and best-selling product)

Reverse Polish Notation (RPN)

Explanation

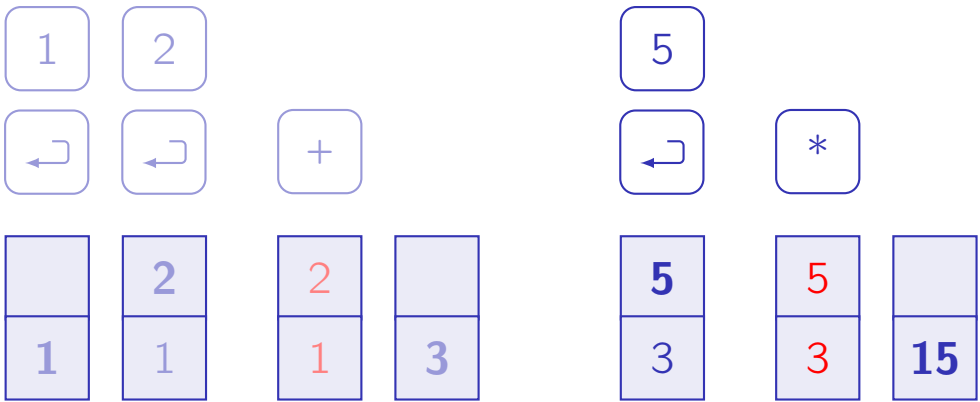
$$1 + 2$$



Reverse Polish Notation (RPN)

Explanation

$$5 \cdot (1 + 2)$$



Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```python
from rpncalc.utils import calc, Config

class RPNCalculator:
    def __init__(self, config):
        self.config = config
        self.stack = []

    def get_inputs(self) -> list[str]:
        inp = input(self.config.prompt + " ")
        return inp.split()

    def run(self) -> None:
        while True:
            for inp in self.get_inputs():
```

```python
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```python
from rpncalc.utils import calc, Config

class RPNCalculator:
    def __init__(self, config):
        self.config = config
        self.stack = []

    def get_inputs(self) -> list[str]:
        inp = input(self.config.prompt + " ")
        return inp.split()

    def run(self) -> None:
        while True:
            for inp in self.get_inputs():
```

```python
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```python
from rpncalc.utils import calc, Config

class RPNCalculator:
    def __init__(self, config):
        self.config = config
        self.stack = []

    def get_inputs(self) -> list[str]:
        inp = input(self.config.prompt + " ")
        return inp.split()

    def run(self) -> None:
        while True:
            for inp in self.get_inputs():
```

```python
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...
    def run(self) -> None:
        while True:
            for inp in self.get_inputs():
                if inp == "q":
                    return
                elif inp == "p":
                    print(self.stack)
                else:
                    self.evaluate(inp)

    def err(self, msg: str) -> None:
        print(msg, file=sys.stderr)
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...
    def run(self) -> None:
        while True:
            for inp in self.get_inputs():
                if inp == "q":
                    return
                elif inp == "p":
                    print(self.stack)
                else:
                    self.evaluate(inp)

    def err(self, msg: str) -> None:
        print(msg, file=sys.stderr)
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...
    def evaluate(self, inp: str) -> None:
        try:
            self.stack.append(float(inp))
            return
        except ValueError:
            pass

        if inp not in ["+", "-", "*", "/"]:
            self.err(
                f"Invalid input: {inp}")
            return
        ...
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...
    def evaluate(self, inp: str) -> None:
        try:
            self.stack.append(float(inp))
            return
        except ValueError:
            pass

        if inp not in ["+", "-", "*", "/"]:
            self.err(
                f"Invalid input: {inp}")
            return

        ...
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...
    def evaluate(self, inp: str) -> None:
        ...
        if len(self.stack) < 2:
            self.err("Not enough operands")
            return

        b = self.stack.pop()
        a = self.stack.pop()
        ...
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...
    def evaluate(self, inp: str) -> None:
        ...
        try:
            res = calc(a, b, inp)
        except ZeroDivisionError:
            self.err("Division by zero")
            return

        self.stack.append(res)
        print(res)
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...
    def evaluate(self, inp: str) -> None:
        ...
        try:
            res = calc(a, b, inp)
        except ZeroDivisionError:
            self.err("Division by zero")
            return

        self.stack.append(res)
        print(res)
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
if __name__ == "__main__":
    config = Config()
    config.load_cwd()
    rpn = RPNCalculator(config)
    rpn.run()
```

rpncalc/utils.py

```
class Config:
    def __init__(self, prompt=">"):
        self.prompt = prompt
    ...
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...
```

```
if __name__ == "__main__":
    ...
```

Reverse Polish Notation (RPN)

rpncalc/rpn_v2.py

```
if __name__ == "__main__":
    config = Config()
    config.load_cwd()
    rpn = RPNCalculator(config)
    rpn.run()
```

rpncalc/utils.py

```
class Config:
    def __init__(self, prompt=">"):
        self.prompt = prompt
    ...
```

```
class RPNCalculator:
    def __init__(self, ...):
        ...

    def get_inputs(self):
        ...

    def run(self):
        ...

    def err(self, msg):
        ...

    def evaluate(self, inp):
        ...


if __name__ == "__main__":
    ...
```



Property-based testing with Hypothesis

pyte.st/hypothesis 

Motivation

From the Hypothesis website, `https://hypothesis.works`:

> *Most testing is ineffective*
>
> *Normal "automated" software testing is surprisingly manual.*
> *Every scenario the computer runs, someone had to write by hand.*
> *Hypothesis can fix this.*

Property-based testing with Hypothesis



Motivation

From the Hypothesis website, `https://hypothesis.works`:

> *Most testing is ineffective*
>
> *Normal "automated" software testing is surprisingly manual.*
> *Every scenario the computer runs, someone had to write by hand.*
> *Hypothesis can fix this.*

Idea behind "Property-Based testing" (popularized by QuickCheck/Haskell):

- Generate input data based on a *strategy* (how should the data look?)
- Run the test case many times (say, 100), with different random generated data
- Check for *properties* / *invariants* that should hold for all input data
- If there is a failure, *minimize* the input data to a minimal failing example

Property-based testing with Hypothesis

Old buggy RPN calculator

rpncalc/rpn_v1.py

```
class RPNCalculator:
    ...

    def evaluate(self, inp: str):
        if inp.isdigit():
            n = float(inp)
            self.stack.append(n)
        elif inp in "+-*/":
            b = self.stack.pop()
            a = self.stack.pop()
            res = calc(a, b, inp)
            self.stack.append(res)
            print(res)
```

```
> 1
> +
Traceback (most recent call last):
  ...
  File ..., in <module>
    rpn.run()
  File ..., in run
    self.evaluate(inp)
  File ..., in evaluate
    a = self.stack.pop()
IndexError: pop from empty list
```

Property-based testing with Hypothesis

pyte.st/hypothesis-start



Old buggy RPN calculator: Hypothesis test

plugins/test_hypothesis_rpncalc_v1.py

```
from hypothesis import given, strategies as st
from rpncalc.rpn_v1 import RPNCalculator

@given(st.text())
def test_random_strings(s: str):
    rpn = RPNCalculator()
    rpn.evaluate(s)
```

Property-based testing with Hypothesis

pyte.st/hypothesis-start



Old buggy RPN calculator: Hypothesis test

plugins/test_hypothesis_rpncalc_v1.py

```python
from hypothesis import given, strategies as st
from rpncalc.rpn_v1 import RPNCalculator

@given(st.text())
def test_random_strings(s: str):
    rpn = RPNCalculator()
    rpn.evaluate(s)
```

```
>   b = self.stack.pop()
E   IndexError: pop from empty list
E   Falsifying example:
E       test_random_strings(s='')
```

Property-based testing with Hypothesis

pyte.st/hypothesis-start



Old buggy RPN calculator: Hypothesis test

plugins/test_hypothesis_rpncalc_v1.py

```
from hypothesis import given, strategies as st
from rpncalc.rpn_v1 import RPNCalculator

@given(st.text())
def test_random_strings(s: str):
    rpn = RPNCalculator()
    rpn.evaluate(s)
```

```
elif inp in "+-*/":
    ...
```

```
>   b = self.stack.pop()
E   IndexError: pop from empty list
E   Falsifying example:
E       test_random_strings(s='')
```

```
>>> "" in "+-*/"
True
```

Property-based testing with Hypothesis

Surely v2 is bug-free?

rpncalc/rpn_v2.py

```
class RPNCalculator:
    ...

    def evaluate(self, inp: str) -> None:
        ...
        if len(self.stack) < 2:
            self.err("Not enough operands")
            return

        b = self.stack.pop()
        a = self.stack.pop()
```

Property-based testing with Hypothesis

Surely v2 is bug-free?

rpncalc/rpn_v2.py

```python
class RPNCalculator:
    ...

    def evaluate(self, inp: str) -> None:
        ...
        if len(self.stack) < 2:
            self.err("Not enough operands")
            return

        b = self.stack.pop()
        a = self.stack.pop()
```

```
test_random_strings
PASSED
```

Property-based testing with Hypothesis

Surely v2 is bug-free?

plugins/test_hypothesis_rpncalc_v2.py

```python
@given(st.integers(), st.integers())
def test_operators(n1: int, n2: int):
    rpn = RPNCalculator(Config())
    rpn.evaluate(str(n1))
    rpn.evaluate(str(n2))
    rpn.evaluate("+")
    assert rpn.stack == [n1 + n2]
```

Property-based testing with Hypothesis

Surely v2 is bug-free?

plugins/test_hypothesis_rpncalc_v2.py

```
@given(st.integers(), st.integers())
def test_operators(n1: int, n2: int):
    rpn = RPNCalculator(Config())
    rpn.evaluate(str(n1))
    rpn.evaluate(str(n2))
    rpn.evaluate("+")
    assert rpn.stack == [n1 + n2]
E   assert [1.055531162664984 8 e+16] == [1055531162664984 9 ]
E   ...
E   Falsifying example: test_operators(
E       n1=0,
E       n2=10555311626649849,
E   )
```

Property-based testing with Hypothesis

Surely v2 is bug-free?

plugins/test_hypothesis_rpncalc_v2.py

```
@given(st.integers(), st.integers())
def test_operators(n1: int, n2: int):
    rpn = RPNCalculator(Config())
    rpn.evaluate(str(n1))
    rpn.evaluate(str(n2))
    rpn.evaluate("+")
    assert rpn.stack == [n1 + n2]
E   assert [1.055531162664984 8 e+16] == [1055531162664984 9 ]
E   ...
E   Falsifying example: test_operators(
E       n1=0,
E       n2=10555311626649849,
E   )
```

```
> 10555311626649849
> 0
> +
1.055531162664984 8 e+16
```

Property-based testing with Hypothesis

Surely v2 is bug-free?

plugins/test_hypothesis_rpncalc_v2.py

```python
@given(st.integers(), st.integers())
def test_operators(n1: int, n2: int):
    rpn = RPNCalculator(Config())
    rpn.evaluate(str(n1))
    rpn.evaluate(str(n2))
    rpn.evaluate("+")
    assert rpn.stack == [n1 + n2]
```

```
E   assert [1.055531162664984 8 e+16] == [1055531162664984 9 ]
E   ...
E   Falsifying example: test_operators(
E       n1=0,
E       n2=10555311626649849,
E   )
```

> 10555311626649849
> 0
> +
1.055531162664984 8 e+16

rpncalc/rpn _v2 .py

```
...
self.stack.append( float(inp) )
```

Property-based testing with Hypothesis

pyte.st/hypothesis-tut



More sophisticated strategies

```
@given(
    st.lists(
        st.one_of(
            st.integers().map(str),
            st.floats().map(str),
            st.just("+"), st.just("-"),
            st.just("*"), st.just("/"),
        )
    )
)
def test_usage(inputs: list[str]):
    rpn = RPNCalculator(Config())
    for inp in inputs:
        rpn.evaluate(inp)
```

Property-based testing with Hypothesis

pyte.st/hypothesis-tut



More sophisticated strategies

```python
@given(
    st.lists(
        st.one_of(
            st.integers().map(str),
            st.floats().map(str),
            st.just("+"), st.just("-"),
            st.just("*"), st.just("/"),
        )
    )
)
def test_usage(inputs: list[str]):
    rpn = RPNCalculator(Config())
    for inp in inputs:
        rpn.evaluate(inp)
```

```
Trying example: test_usage(
  inputs=[
    '-2.2250738585e-313',
    '/', '-10', '110', '+', '+'])
Not enough operands
100.0
100.0
```

Property-based testing with Hypothesis

pyte.st/hypothesis-tut



More sophisticated strategies

```
@given(
    st.lists(
        st.one_of(
            st.integers().map(str),
            st.floats().map(str),
            st.just("+"), st.just("-"),
            st.just("*"), st.just("/"),
        )
    )
)
def test_usage(inputs: list[str]):
    rpn = RPNCalculator(Config())
    for inp in inputs:
        rpn.evaluate(inp)
```

```
Trying example: test_usage(
  inputs=[
    '-2.2250738585e-313',
    '/', '-10', '110', '+', '+'])
Not enough operands
100.0
100.0

Trying example: test_usage(
  inputs=['+', '+', '-', '*'],
)
Not enough operands
Not enough operands
Not enough operands
Not enough operands
```

Property-based testing with Hypothesis

Another example: Run length encoding

Original:

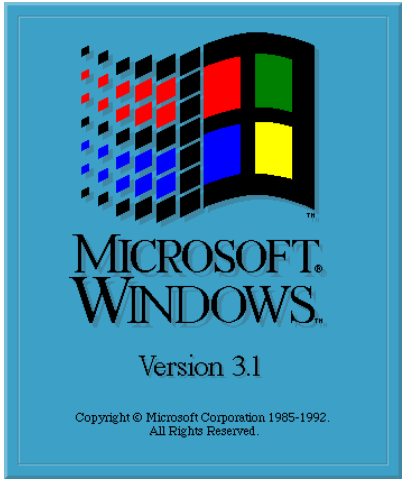
PPPPPYYYPPPPPPPYYYY

Compressed:

5P3Y7P4Y

Used in:

- Fax machines
- Windows 3.1 startup screen
- ...



Property-based testing with Hypothesis

Decoding

input: `[(5, "P"), (3, "Y"), (7, "P"), (4, "Y")]`
output: `"PPPPPYYYPPPPPPPYYYY"`

plugins/test_hypothesis.py

```python
def decode(inp: list[tuple[int, str]]) -> str:
    out = ""
    for count, character in inp:
        out += count * character
    return out
```

```
""
" PPPPP "
"PPPPP YYY "
"PPPPPYYY PPPPPPP "
"PPPPPYYYPPPPPPP YYYY "
```

Property-based testing with Hypothesis

Encoding

```python
def encode(input_string: str) -> list[tuple[int, str]]:
    count = 1; prev = ""; out = []
    for character in input_string:
        if character != prev:
            if prev:
                entry = (count, prev)
                out.append(entry)
            count = 1
            prev = character
        else:
            count += 1
    entry = (count, character)
    out.append(entry)
    return out
```

input:

```
"PPPPPYYYPPPPPPPYYYY"
```

output:

```
[
    (5, "P"),
    (3, "Y"),
    (7, "P"),
    (4, "Y"),
]
```

Property-based testing with Hypothesis

Encoding

```python
def encode(input_string: str) -> list[tuple[int, str]]:
    count = 1; prev = ""; out = []
    for character in input_string:
        if character != prev:
            if prev:
                entry = (count, prev)
                out.append(entry)
            count = 1
            prev = character
        else:
            count += 1
    entry = (count, character)
    out.append(entry)
    return out
```

input:

```
"PPPPPYYYPPPPPPPYYYY"
```

output:

```
[
    (5, "P"),
    (3, "Y"),
    (7, "P"),
    (4, "Y"),
]
```

Property-based testing with Hypothesis

Encoding

```python
def encode(input_string: str) -> list[tuple[int, str]]:
    count = 1; prev = ""; out = []
    for character in input_string:
        if character != prev:
            if prev:
                entry = (count, prev)
                out.append(entry)
            count = 1
            prev = character
        else:
            count += 1
    entry = (count, character)
    out.append(entry)
    return out
```

input:

```
"PPPPPYYYPPPPPPPYYYY"
```

output:

```
[
    (5, "P"),
    (3, "Y"),
    (7, "P"),
    (4, "Y"),
]
```

Property-based testing with Hypothesis

Encoding

```python
def encode(input_string: str) -> list[tuple[int, str]]:
    count = 1; prev = ""; out = []
    for character in input_string:
        if character != prev:
            if prev:
                entry = (count, prev)
                out.append(entry)
            count = 1
            prev = character
        else:
            count += 1
    entry = (count, character)
    out.append(entry)
    return out
```

input:

```
"PPPPPYYYPPPPPPPYYYY"
```

output:

```
[
    (5, "P"),
    (3, "Y"),
    (7, "P"),
    (4, "Y"),
]
```

Property-based testing with Hypothesis

Encoding

```
def encode(input_string: str) -> list[tuple[int, str]]:
    count = 1; prev = ""; out = []
    for character in input_string:
        if character != prev:
            if prev:
                entry = (count, prev)
                out.append(entry)
            count = 1
            prev = character
        else:
            count += 1
    entry = (count, character)
    out.append(entry)
    return out
```

input:

```
"PPPPPYYYPPPPPPPYYYY"
```

output:

```
[
    (5, "P"),
    (3, "Y"),
    (7, "P"),
    (4, "Y"),
]
```

Property-based testing with Hypothesis

Encoding

```python
def encode(input_string: str) -> list[tuple[int, str]]:
    count = 1; prev = ""; out = []
    for character in input_string:
        if character != prev:
            if prev:
                entry = (count, prev)
                out.append(entry)
            count = 1
            prev = character
        else:
            count += 1
    entry = (count, character)
    out.append(entry)
    return out
```

input:

```
"PPPPPYYYPPPPPPPYYYY"
```

output:

```
[
    (5, "P"),
    (3, "Y"),
    (7, "P"),
    (4, "Y"),
]
```

Property-based testing with Hypothesis

Manual test

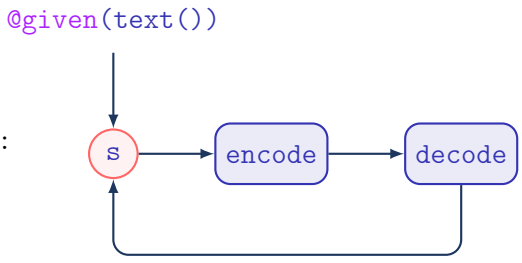
```python
@pytest.mark.parametrize("inp, out", [
    ("PYY", [(1, "P"), (2, "Y")]),
    ...
])
def test_rle(inp: str, out: list[tuple[int, str]]):
    assert encode(inp) == out
```

Can we think of all corner-cases?
Maybe it's easier to think of *invariants* (certain properties that always hold true)?

Property-based testing with Hypothesis

Test with Hypothesis

```python
@given(text())
def test_decode_inverts_encode(s: str):
    assert decode(encode(s)) == s
```



Property-based testing with Hypothesis

Result

```
UnboundLocalError:
local variable 'character'
referenced before assignment

Falsifying example:
test_decode_inverts_encode(s='')
```

```
def encode(input_string: str) -> str:
    count = 1; prev = ""; out = []
    for character in input_string:
        ...
    entry = (count, character)
    out.append(entry)
    return out
```

Property-based testing with Hypothesis

```python
def encode(input_string: str):
    if not input_string:
        return []
    count = 1; prev = ""; out = []
    for character in input_string:
        if character != prev:
            if prev:
                entry = (count, prev)
                out.append(entry)
            count = 1
            prev = character
        else:
            count += 1
    entry = (count, character)
    out.append(entry)
    return out
```

Demo:: hypothesis

`plugins/test_hypothesis.py`

- Fix the issue, e.g. like shown.
- Re-run the test, it should pass. Observe examples via `-s` and `--hypothesis-verbosity=verbose`.
- Deliberately introduce a more subtle issue, e.g. break resetting the count by commenting out the second `count = 1`.
- Re-run the test – you should get a minimal example which fails the test.

Property-based testing with Hypothesis

Another possibility

```
def encode_itertools(input_string: str) -> list[tuple[int, str]]:
    return [
        (len(list(group)), character)
        for character, group
        in itertools.groupby(input_string)
    ]


@given(text())
def test_alternative_implementation(s: str):
    assert encode_itertools(s) == encode(s)
```

Property-based testing with Hypothesis

What it can do

- Generate data based on types/strategies
- Combine/filter various strategies to generate more sophisticated data
- Generate data matching a Django model
- Generate Numpy arrays, Pandas dataframes, etc.
- Generate data matching a syntax grammar
- Generate test actions (rather than data) based on a state machine

Building a compiler at university

Intro

In 2019, as part of my studies, a classmate and I built a custom compiler/runtime for a C#-like programming language:

```
class Program {
    void Main() {
        int i;
        while (i < 10) {
            WriteInt(Fibonacci(i)); WriteString("");
            i = i + 1;
        }
    }
    int Fibonacci(int n) {
        if (n == 0 || n == 1) { return n; }
        return Fibonacci(n - 1) + Fibonacci(n - 2);
    }
}
```

Building a compiler at university

EBNF

EBNF (Extended Backus-Naur Form) is a notation for specifying the grammar/syntax of programming languages. As part of the work assignment, we got a specification:

```
class : "class" " " identifier [ ":" identifier ]
        "{" ( classmember )* "}"
...
statementblock : "{" ( statement )* "}"
statement : ";" | variable | assignment | ifstatement | whilestatement
            | callstatement | returnstatement
assignment : designator "=" expression ";"
ifstatement : "if" "(" expression ")" statementblock
              [ "else" statementblock ]
...
```

Building a compiler at university

Hypothesis & Lark

```python
import lark
import hypothesis
import hypothesis.extra.lark

GRAMMAR = r"""..."""
PARSER = lark.Lark(GRAMMAR, start="program")

@hypothesis.given(
    hypothesis.extra.lark.from_lark(PARSER, start="ifstatement")
)
def test_programs(code: str):
    # Run the custom parser/lexer and make sure there is no syntax error
    print(code)
```

Building a compiler at university

Hypothesis & Lark: Result

```
if (0) {}                    if (0 && 0 || 0) {} else {}

if ((! " " / 00 * '\\' != '\\' cast A[][] && M.a) / 20 cast aAa[][][]) {
} else {
}

if (v7 % - f8 - +new l()) {} else { return; }

if (
    (new p() - new b0[A.A - new b0[A.A + 0] * 0] * 0 !=
    0x0 * 0 cast a || 0xa) is Aa == (new p() - new b0[A.A + 0] * 0 !=
    0x0 * 0 cast a || 0xa) || 0xa
) {
}
```

Building a compiler at university

Outcome

- My classmate wrote the compiler (in C#),
  I wrote lots of tests and told him about the bugs I found.
- No exam, the code we handed in was graded
  (both manually and based on hidden testcases)
- We got the best possible grade *and* found bugs in the template code from the professor (which was used by hundreds of students in previous years)
- Thanks pytest, thanks Hypothesis!

Upcoming events

- **July 20th – 24th 2026**
  pytest development sprint
  pytest-dev/sprint
  Klaus, Vorarlberg, Austria

- **September 9th 2026**
  CH Open Workshoptage (workshoptage.ch)
  Python Best Practices
  Rapperswil, Switzerland

- **March 9th – 11th 2027**
  Python Academy (python-academy.com)
  Professional Testing with Python (3 days)
  Leipzig, Germany & Remote

- **Custom training / coaching:**
  - Python
  - pytest
  - GUI programming with Qt
  - Best Practices
    (packaging, linting, etc.)
  - Git
  - ...

  Remote or on-site
  freya@bruhin.software
  https://bruhin.software/

BRUHIN SOFTWARE

Feedback and questions

Freya Bruhin
freya@bruhin.software
https://bruhin.software
@The-Compiler

linkedin.com/in/freya-bruhin
@the_compiler@mastodon.social
@the-compiler.org

BRUHIN
SOFTWARE

The-Compiler/hypothesis-talk

Copyright 2015 – 2026 Freya Bruhin

CC BY 4.0